

TP n°4 : MapReduce (Langage Java)

Matière : Introduction au Big Data

Classe : 1ère année MPSD

Enseignant : Mohamed Anouar DAHDEH



Objectifs:

Après avoir terminé ce TP, vous serez en mesure de :

- Créer un projet **MapReduce** avec Eclipse
- Créer un programme **MapReduce** en java
- Compiler et déboguer votre programme **MapReduce** sous Eclipse
- Exécutez votre programme **MapReduce**, surveillez les travaux et visualisez la sortie du journal sous **Hadoop**.

Introduction

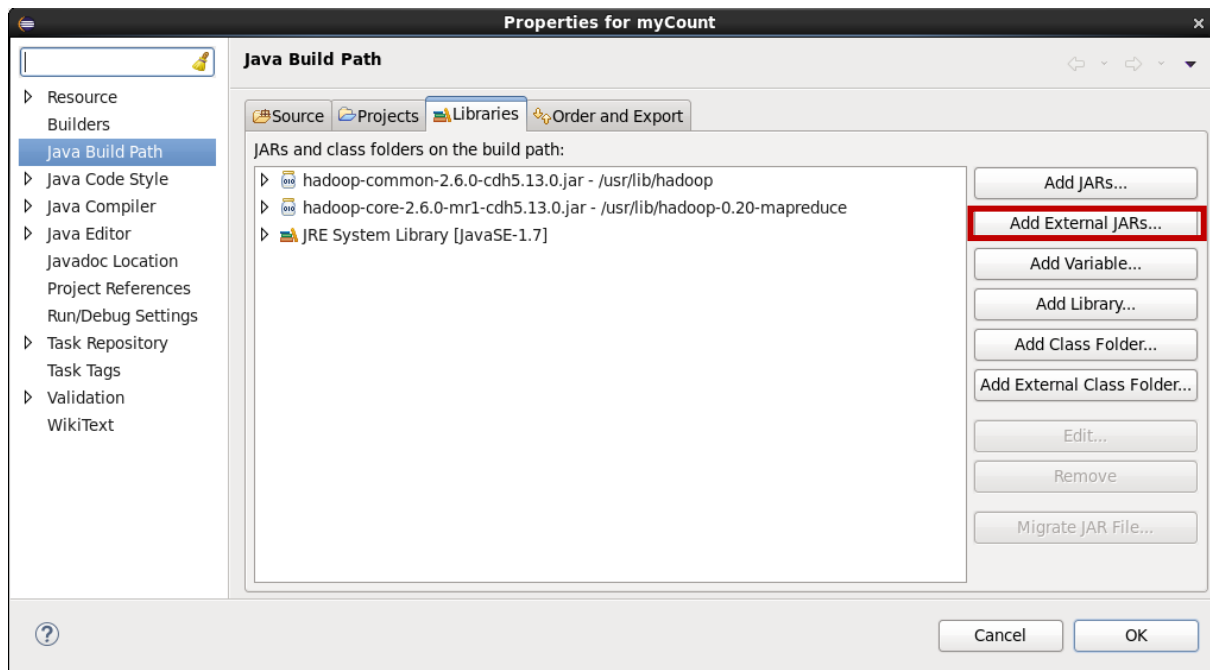
Compter le nombre de mots dans n'importe quelle langage de programmation est un jeu d'enfant comme en C, C ++, Python, Java, etc. **MapReduce** utilise également Java, mais il est très facile si vous connaissez la syntaxe pour l'écrire. Vous allez d'abord apprendre à exécuter ce code similaire au programme «Hello World» dans d'autres langages. Alors voici les étapes qui montrent comment écrire un code **MapReduce** pour **WordCount**.

1- Création d'un projet Java avec Eclipse

- a- Lancer **Eclipse** sous **Cloudera VM**
- b- Créer un nouveau projet java (**Java Project**) sous le nom **WordCount**
- c- Créer un package dans ce projet : **com.fsb.mapreduce**
- d- Créer trois classes dans ce package :
 - **WCDriver** (contenant la fonction main)
 - **WCMapper** (décrit la fonction map)
 - **WCReducer** (décrit la fonction reduce)
- e- Ajout des dépendances et bibliothèques de **hadoop** écosystèmes :
Clic droit sur le projet → **Build Path** → **Configure Build Path**

Dans la figure suivante, vous pouvez voir l'option Ajouter des fichiers JAR externes sur le côté droit. Cliquez dessus et ajoutez les fichiers mentionnés ci-dessous. Vous trouvez ces fichier dans `/usr/lib/` :

1. `/usr/lib/hadoop-0.20-mapreduce/hadoop-core-2.6.0-mr1-cdh5.13.0.jar`
2. `/usr/lib/hadoop/hadoop-common-2.6.0-cdh5.13.0.jar`



2- Création du Mapper

Copier le code suivant dans la classe **WCMapper** :

```

// Importing libraries
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;

public class WCMapper extends MapReduceBase implements Mapper<LongWritable,
    Text, Text, IntWritable> {

    // Map function
    public void map(LongWritable key, Text value, OutputCollector<Text,
        IntWritable> output, Reporter rep) throws IOException
    {
        String line = value.toString();

        // Splitting the line on spaces
        for (String word : line.split(" "))
        {
            if (word.length() > 0)
            {
                output.collect(new Text(word), new IntWritable(1));
            }
        }
    }
}
    
```

Note

Le mappeur crée une paire clé / valeur pour chaque mot, composé du mot et de la valeur **IntWritable**. Utilisez **split(" ")** pour diviser la ligne en mots individuels basés sur des limites de mots. Si l'objet de texte est vide (par exemple, se compose d'un espace blanc), allez à l'objet suivant analysé. Sinon, écrire une paire à l'objet de contexte (**OutputCollector**) pour le travail clé / valeur.

3- Création du Reducer

Copier le code suivant dans la classe **WCReducer** :

```
// Importing libraries
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

public class WCReducer extends MapReduceBase implements Reducer<Text,
    IntWritable, Text, IntWritable> {

    // Reduce function
    public void reduce(Text key, Iterator<IntWritable> value,
        OutputCollector<Text, IntWritable> output,
        Reporter rep) throws IOException
    {

        int count = 0;

        // Counting the frequency of each words
        while (value.hasNext())
        {
            IntWritable i = value.next();
            count += i.get();
        }

        output.collect(key, new IntWritable(count));
    }
}
```

Note

Le processus du réducteur consiste à ajouter un au comptage pour le mot courant dans la paire clé / valeur pour le nombre global de ce mot de tous mappeurs. Il écrit ensuite le résultat de ce mot à l'objet de contexte réducteur, et passe à la suivante. Lorsque toutes les paires clé / valeur intermédiaires sont traitées, la map / reduce tâche est terminée. L'application enregistre les résultats à l'emplacement de sortie dans HDFS.

4- Création de la classe principale

Copier le code suivant dans la classe **WCDriver** :

```
// Importing libraries
import java.io.IOException;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WCDriver extends Configured implements Tool {

    public int run(String args[]) throws IOException
    {
        if (args.length < 2)
        {
            System.out.println("Please give valid inputs");
            return -1;
        }

        JobConf conf = new JobConf(WCDriver.class);
        FileInputFormat.setInputPaths(conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));
        conf.setMapperClass(WCMapper.class);
        conf.setReducerClass(WCReducer.class);
        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);
        JobClient.runJob(conf);
        return 0;
    }

    // Main Method
    public static void main(String args[]) throws Exception
    {
        int exitCode = ToolRunner.run(new WCDriver(), args);
        System.out.println(exitCode);
    }
}
```

Note

Utilisez la classe Path pour accéder à des fichiers dans HDFS. Dans vos instructions de configuration d'emploi, vous passez les chemins nécessaires en utilisant les classes FileInputFormat et FileInputFormat.

```
FileInputFormat.addInputPath(job, new Path(args[0]));
```

```
FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

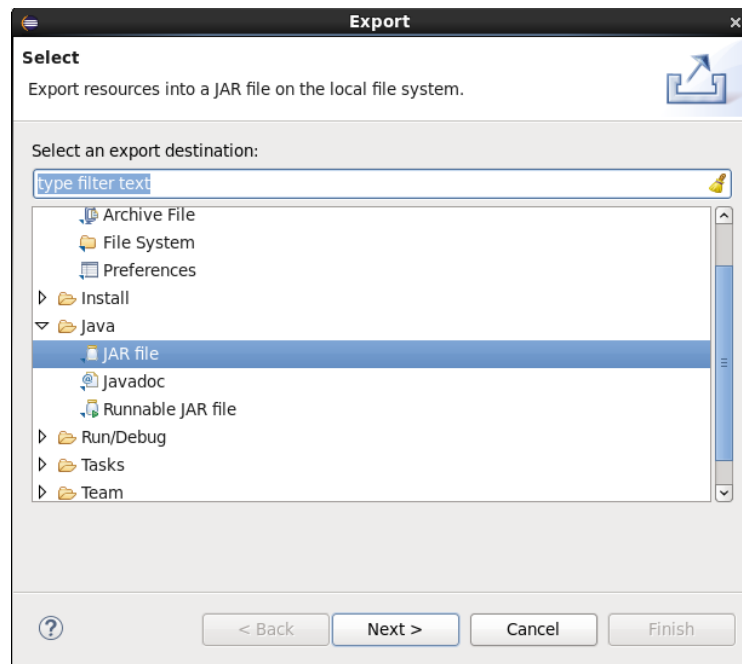
Définissez la classe de Map et la classe Reducer pour le travail. Dans ce cas, utilisez la WCMapper et WCReducer :

```

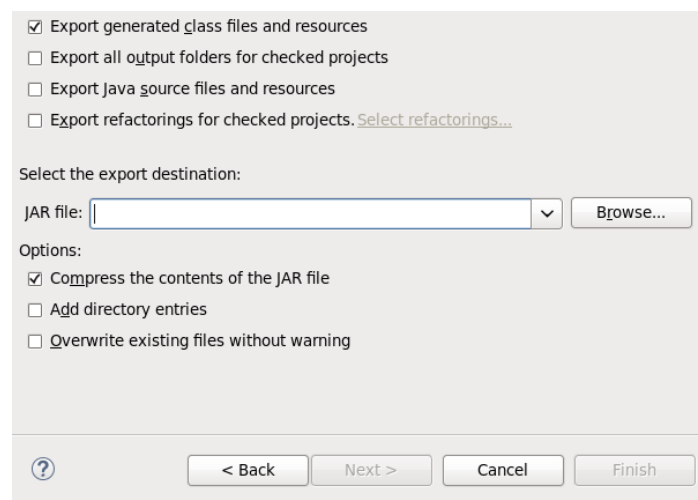
job.setMapperClass(WCMapper.class);
job.setReducerClass(WCReducer.class);
    
```

5- Génération du fichier Jar

Pour exécuter le programme WordCount sous **hadoop**, il faut le compiler et générer un fichier jar. Clic droit sur le projet → **Export** → sélectionner **JAR file**



Renommer le fichier **jar** (WordCount.jar) et enregistrer le dans le dossier /home/cloudera → Clic sur **next** → .. → Clic sur **Finish**.



Vérifier la génération du fichier JAR dans /home/cloudera

6- Création des fichiers Input

Lancer **gedit** et taper les textes suivants dans chacun des fichiers nommés File0, File1, File2

File0 : J'apprend Hadoop

File1 : Mon premier programme Hadoop MapReduce

File2 : Le programme MapReduce est facile

Enregistrer les fichiers dans le répertoire /home/cloudera.

6.1- Transfert des fichiers vers HDFS

- Lancer un terminal
- Créer les répertoires nécessaires :

```
/user/cloudera/wordcount
```

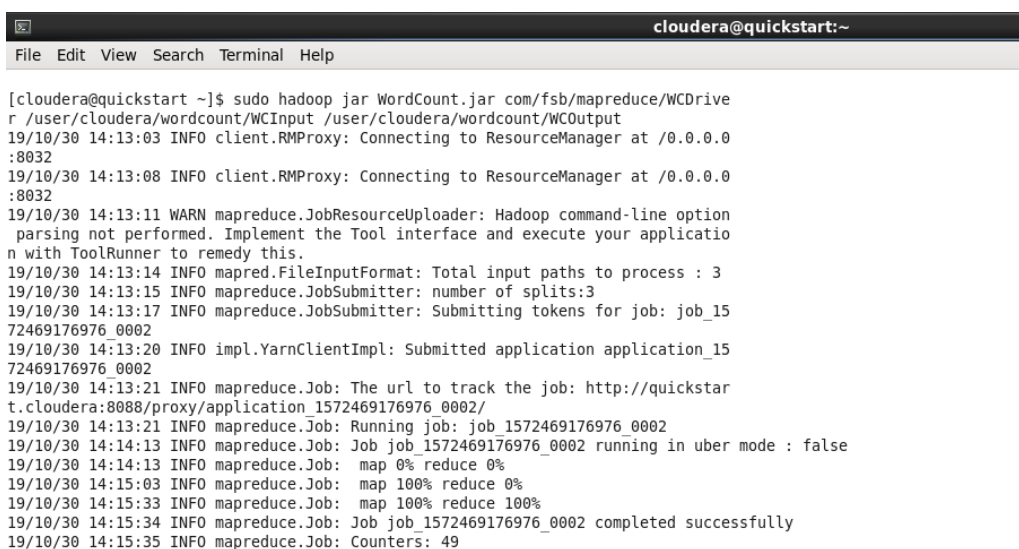
```
/user/cloudera/wordcount/WCInput
```

- Copier les fichiers File0, File1 et File2 sous **HDFS**
- Vérifier le résultat du transfert des fichiers avec **Hue**

7- Exécution du programme sous Hadoop

Pour exécuter le programme MapReduce « **WordCount** », on doit se déplacer dans le dossier /home/cloudera et invoquer l'instruction suivante en spécifiant à **hadoop** le programme jar, le chemin de repertoire contenant les données d'entrée et le chemin de repertoire où le programme va stocker le resultat d'exécution.

```
[cloudera@quickstart~]$ hadoop jar WordCount.jar  
com/fsb/mapreduce/WCDriver /user/cloudera/wordcount/WCInput  
/user/cloudera/wordcount/WCOutput
```



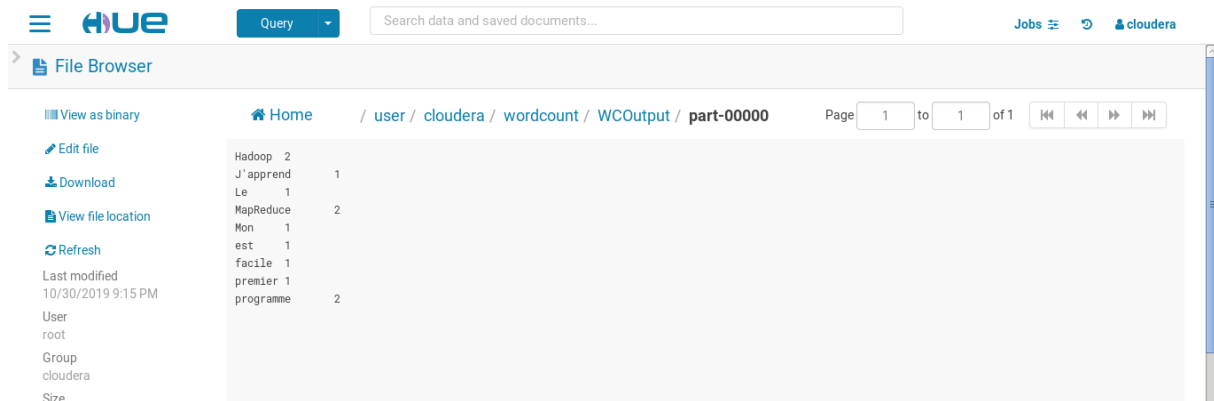
```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
[cloudera@quickstart ~]$ sudo hadoop jar WordCount.jar com/fsb/mapreduce/WCDrive  
r /user/cloudera/wordcount/WCInput /user/cloudera/wordcount/WCOutput  
19/10/30 14:13:03 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0  
:8032  
19/10/30 14:13:08 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0  
:8032  
19/10/30 14:13:11 WARN mapreduce.JobResourceUploader: Hadoop command-line option  
parsing not performed. Implement the Tool interface and execute your applicatio  
n with ToolRunner to remedy this.  
19/10/30 14:13:14 INFO mapred.FileInputFormat: Total input paths to process : 3  
19/10/30 14:13:15 INFO mapreduce.JobSubmitter: number of splits:3  
19/10/30 14:13:17 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_15  
72469176976_0002  
19/10/30 14:13:20 INFO impl.YarnClientImpl: Submitted application application_15  
72469176976_0002  
19/10/30 14:13:21 INFO mapreduce.Job: The url to track the job: http://quickstar  
t.cloudera:8088/proxy/application_1572469176976_0002/  
19/10/30 14:13:21 INFO mapreduce.Job: Running job: job_1572469176976_0002  
19/10/30 14:14:13 INFO mapreduce.Job: Job job_1572469176976_0002 running in uber mode : false  
19/10/30 14:14:13 INFO mapreduce.Job: map 0% reduce 0%  
19/10/30 14:15:03 INFO mapreduce.Job: map 100% reduce 0%  
19/10/30 14:15:33 INFO mapreduce.Job: map 100% reduce 100%  
19/10/30 14:15:34 INFO mapreduce.Job: Job job_1572469176976_0002 completed successfully  
19/10/30 14:15:35 INFO mapreduce.Job: Counters: 49
```

8- Visualisation du résultat

Pour visualiser le résultat :

- Afficher le contenu du dossier WCOOutput
- Combien de fichiers ?
- Afficher ces fichiers.
- Vérifier le résultat avec Hue

Vous devriez avoir ce résultat



The screenshot shows the Hue File Browser interface. The breadcrumb path is: Home / user / cloudera / wordcount / WCOOutput / part-00000. The file list shows the following contents:

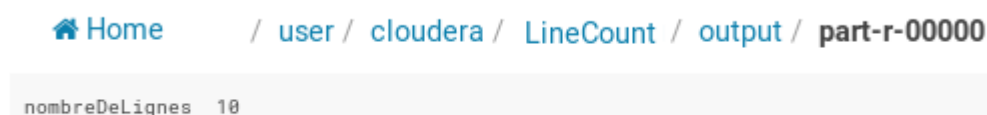
File Name	Size
Hadoop	2
J'apprend	1
Le	1
MapReduce	2
Mon	1
est	1
facile	1
premier	1
programme	2

Additional details visible in the interface include: 'View as binary', 'Edit file', 'Download', 'View file location', 'Refresh', 'Last modified: 10/30/2019 9:15 PM', 'User: root', 'Group: cloudera', and 'Size'.

Travail à faire :

Ecrivez le programme MapReduce «**LineCount**», qui permet de compter le nombre de lignes du fichier ReadMe.txt

Vous devriez avoir le résultat suivant :



The screenshot shows the Hue File Browser interface with the breadcrumb path: Home / user / cloudera / LineCount / output / part-r-00000. The output content is displayed as:

```
nombreDeLignes 10
```